the same as in the previous version of ASP.NET. They're also used for purposes other than validation. Entity Framework Core (among others) uses `DataAnnotations` to define the types of columns and rules to use when creating database tables from C# classes. You can read more about Entity Framework Core in chapter 12 and in *Entity Framework Core in Action*, 2nd ed., by Jon P. Smith (Manning, 2021).

If the `DataAnnotation` attributes provided out of the box don't cover everything you need, it's possible to write custom attributes by deriving from the base `ValidationAttribute`. You'll see how to create a custom validation attribute in chapter 32.

One common limitation with `DataAnnotation` attributes is that it's hard to validate properties that depend on the values of other properties. Maybe the `UserModel` type from listing 7.10 requires you to provide either an email address or a phone number but not both, which is hard to achieve with attributes. In this type of situation, you can implement `IValidatableObject` in your models instead of, or in addition to, using attributes. In listing 7.11, a validation rule is added to `UserModel` whether the email or phone number is provided. If it isn't, `Validate()` returns a `ValidationResult` describing the problem.

Listing 7.11 Implementing `IValidatableObject`

```
using System.ComponentModel.DataAnnotations;

public record CreateUserModel : IValidatableObject                ❶
{
    [EmailAddress]                                                ❷
    public string Email { get; set; }

    [Phone]                                                       ❷
    public string PhoneNumber { get; set; }
```